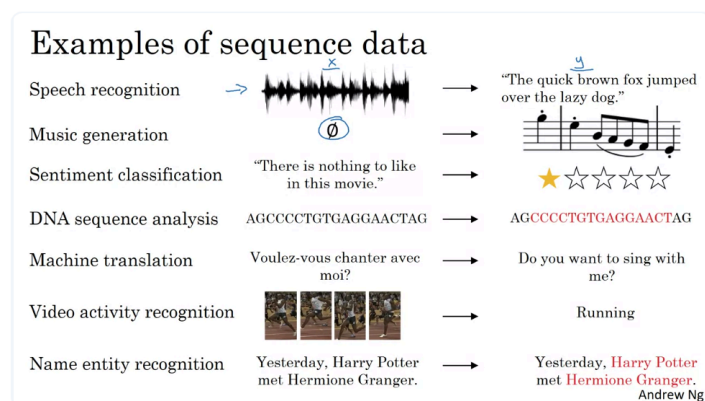


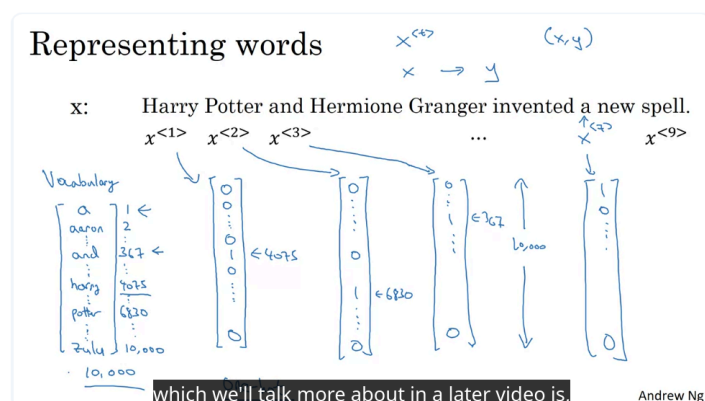
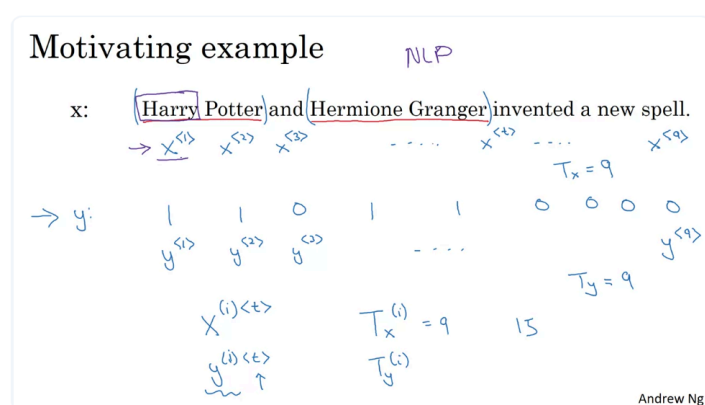
Recurrent Neural Networks Part 1

Why sequence Models

Many data is sequential! → Need sequence model to handle these types of data



Notation



Sequence Models: Notation and Word Representation

1. Motivating Example: Named-Entity Recognition

Sequence models can be applied to many different tasks. One example is processing a sentence such as:

Harry Potter and Hermione Granger invented a new spell.

The goal is to build a sequence model that automatically identifies the names of people in the sentence.

This task is called **Named-Entity Recognition**, commonly abbreviated as **NER**.

Named-entity recognition systems can identify several types of entities in text, including:

- People's names
- Company names
- Times
- Locations
- Country names

- Currency names

For example, search engines can use named-entity recognition to process recent news articles and index the people mentioned in those articles.

In the example sentence:

- "Harry Potter" is a person's name.
- "Hermione Granger" is a person's name.

The input sentence is represented by x , and the model produces an output sequence y .

The output contains one prediction for each input word. Each output indicates whether the corresponding word is part of a person's name.

A simplified output representation is:

Input word	Part of a person's name
Harry	1
Potter	1
and	0
Hermione	1
Granger	1
invented	0
a	0
new	0
spell	0

Here:

- 1 means that the word is part of a person's name.
- 0 means that the word is not part of a person's name.

This is not necessarily the best possible output representation.

A more sophisticated representation could also indicate:

- Where a person's name starts
- Where a person's name ends
- Which words belong to the same name

For example, the model could identify that:

- "Harry Potter" starts at the first word and ends at the second word.
- "Hermione Granger" starts at the fourth word and ends at the fifth word.

However, the lecture uses the simpler binary output representation as a motivating example.

2. Input Sequence Notation

The input sentence contains nine words.

Each word is represented as an element in the input sequence:

$x\{1\}$, $x\{2\}$, $x\{3\}$, ..., $x\{9\}$

For the example sentence:

- $x\{1\}$ represents "Harry"
- $x\{2\}$ represents "Potter"
- $x\{3\}$ represents "and"
- $x\{4\}$ represents "Hermione"
- $x\{5\}$ represents "Granger"
- $x\{6\}$ represents "invented"

- $x\langle 7 \rangle$ represents "a"
- $x\langle 8 \rangle$ represents "new"
- $x\langle 9 \rangle$ represents "spell"

The notation $x\langle t \rangle$ refers to the input at position t in the sequence.

The index t is often associated with time in temporal sequences. However, it is also used to represent positions in sequences that are not necessarily temporal, such as words in a sentence.

Therefore, t simply indicates a position in the sequence.

3. Output Sequence Notation

The outputs are represented using similar notation:

$y\langle 1 \rangle, y\langle 2 \rangle, y\langle 3 \rangle, \dots, y\langle 9 \rangle$

Each $y\langle t \rangle$ is the output corresponding to $x\langle t \rangle$.

For example:

- $y\langle 1 \rangle = 1$ because "Harry" is part of a person's name.
- $y\langle 2 \rangle = 1$ because "Potter" is part of a person's name.
- $y\langle 3 \rangle = 0$ because "and" is not part of a person's name.
- $y\langle 4 \rangle = 1$ because "Hermione" is part of a person's name.

The input and output elements are aligned by their sequence positions.

4. Input and Output Sequence Lengths

The length of the input sequence is denoted by:

T_x

The length of the output sequence is denoted by:

T_y

In the example sentence, there are nine input words:

$T_x = 9$

There is one output for each input word:

$T_y = 9$

Therefore, in this example:

$T_x = T_y$

However, **the input sequence length and output sequence length do not always have to be equal.**

In other sequence-modeling tasks:

- T_x may be greater than T_y .
- T_y may be greater than T_x .
- T_x may be equal to T_y .

5. Notation for Training Examples

The notation $x^{(i)}$ is used to represent the input of the i -th training example.

Similarly, $y^{(i)}$ represents the output of the i -th training example.

To refer to a specific element at position t in the input sequence of training example i , use: $x^{(i)}\langle t \rangle$

To refer to a specific element at position t in the output sequence of training example i , use: $y^{(i)}\langle t \rangle$

The input sequence length of training example i is written as: $T_x^{(i)}$

The output sequence length of training example i is written as: $T_y^{(i)}$

Different examples in the training set can have different sequence lengths.

For example:

- A sentence with nine words has $T_x^{(i)} = 9$.
- A different sentence with fifteen words has $T_x^{(j)} = 15$.

Therefore, a sequence model must often process examples with variable sequence lengths.

6. Summary of Sequence Notation

Notation	Meaning
$x(t)$	Input element at position t
$y(t)$	Output element at position t
T_x	Length of the input sequence
T_y	Length of the output sequence
$x^{(i)}$	Input of training example i
$y^{(i)}$	Output of training example i
$x^{(i)}(t)$	Input element at position t in training example i
$y^{(i)}(t)$	Output element at position t in training example i
$T_x^{(i)}$	Input sequence length of training example i
$T_y^{(i)}$	Output sequence length of training example i

7. Introduction to Natural Language Processing

This example is an introduction to **Natural Language Processing**, commonly abbreviated as **NLP**.

Natural Language Processing focuses on methods that allow computers to process and understand human language.

To use words as inputs to a neural network, the words must first be converted into numerical representations.

The first step is to create a vocabulary.

8. Vocabulary or Dictionary

A **vocabulary**, also called a **dictionary**, is a list of words used by the model.

Each word in the vocabulary is assigned a unique index.

An example vocabulary may contain:

Vocabulary index	Word
1	a
2	Aaron
367	and
4075	Harry
6830	Potter
10000	Zulu

In this example:

- "a" is word 1.
- "Aaron" is word 2.
- "and" is word 367.
- "Harry" is word 4075.
- "Potter" is word 6830.
- "Zulu" is word 10000.

The example vocabulary contains 10,000 words.

9. Vocabulary Size

A vocabulary size of 10,000 words is relatively small for modern NLP applications.

The lecture mentions several possible vocabulary sizes:

- 10,000 words for a simple illustrative example
- 30,000 to 50,000 words for many commercial applications
- 100,000 words for some applications
- One million words or more for some large internet companies

A vocabulary of 10,000 words is used in the lecture because it is a simple round number that makes the explanation easier.

10. Building a Vocabulary

One way to build a vocabulary is to examine the training set and find the most frequently occurring words.

For a vocabulary of size 10,000, the process may be:

1. Examine all words in the training data.
2. Count how frequently each word appears.
3. Sort the words by frequency.
4. Select the 10,000 most common words.

Another option is to use an online dictionary or word-frequency list containing the most common words in the English language.

The final vocabulary determines which words the model can represent directly.

11. One-Hot Representation

Once a vocabulary has been created, each word can be represented using a **one-hot vector**.

A one-hot vector has the following properties:

- Its dimension is equal to the vocabulary size.
- It contains exactly one value equal to 1.
- All other values are equal to 0.

If the vocabulary contains 10,000 words, each word is represented by a 10,000-dimensional vector.

12. One-Hot Representation of "Harry"

The word "Harry" is located at position 4075 in the vocabulary.

Therefore, $x^{(1)}$ is a vector containing:

- A value of 1 at position 4075
- A value of 0 at every other position

Conceptually:

$$x^{(1)} = [0, 0, \dots, 1, \dots, 0]$$

The single value 1 is located at index 4075.

13. One-Hot Representation of "Potter"

The word "Potter" is located at position 6830 in the vocabulary.

Therefore, $x^{(2)}$ contains:

- A value of 1 at position 6830
- A value of 0 at every other position

Conceptually:

$$x^{(2)} = [0, 0, \dots, 1, \dots, 0]$$

The single value 1 is located at index 6830.

14. One-Hot Representation of "and"

The word "and" is located at position 367 in the vocabulary.

Therefore, $x\langle 3 \rangle$ contains:

- A value of 1 at position 367
- A value of 0 at every other position

Conceptually:

$x\langle 3 \rangle = [0, 0, \dots, 1, \dots, 0]$

The single value 1 is located at index 367.

15. One-Hot Representation of "a"

The word "a" is the first word in the vocabulary.

It appears as the seventh word in the example sentence.

Therefore:

$x\langle 7 \rangle = [1, 0, 0, \dots, 0]$

The value 1 appears in the first position because "a" has vocabulary index 1.

16. Vocabulary Index and Sequence Position

It is important to distinguish between a word's sequence position and its vocabulary index.

For example, the word "Harry" has:

- Sequence position 1 in the sentence
- Vocabulary index 4075

Therefore:

- $x\langle 1 \rangle$ means the first element in the sentence.
- The value 1 inside $x\langle 1 \rangle$ is located at vocabulary position 4075.

Similarly, the word "a" has:

- Sequence position 7 in the sentence
- Vocabulary index 1

Therefore:

- $x\langle 7 \rangle$ means the seventh element in the sentence.
- The value 1 inside $x\langle 7 \rangle$ is located at vocabulary position 1.

These two types of indices represent different concepts.

17. Representing the Entire Sentence

The sentence contains nine words, so it is represented by nine one-hot vectors:

$x\langle 1 \rangle, x\langle 2 \rangle, \dots, x\langle 9 \rangle$

Each vector has 10,000 dimensions because the vocabulary contains 10,000 words.

Each $x\langle t \rangle$ contains exactly one value equal to 1 and zeros everywhere else.

The complete sentence is therefore represented as an ordered sequence of nine one-hot vectors.

18. Learning the Mapping from X to Y

The goal of the sequence model is to learn a mapping:

$X \rightarrow Y$

Here:

- X represents the complete input sequence.

- Y represents the complete target output sequence.

For named-entity recognition, the model receives a sequence of word representations and predicts whether each word is part of a person's name.

The mapping can be written conceptually as:

$$(x\langle 1 \rangle, x\langle 2 \rangle, \dots, x\langle T_x \rangle) \rightarrow (y\langle 1 \rangle, y\langle 2 \rangle, \dots, y\langle T_y \rangle)$$

In the example:

- The input contains nine one-hot vectors.
- The output contains nine labels.
- Each output label corresponds to one input word.

19. Supervised Learning Formulation

The named-entity recognition task is treated as a **supervised learning problem**.

The training data contains both:

- Input sequences X
- Correct target output sequences Y

The model learns from examples in which both the sentence and the correct labels are provided.

For each training example i , the dataset contains:

- $x^{(i)}$: the input sequence
- $y^{(i)}$: the correct output sequence

The model learns parameters that allow it to predict Y from X.

20. Unknown Words

A model may encounter a word that is not included in its vocabulary.

Such a word cannot be represented using its own dedicated vocabulary index.

To handle this situation, a special token called the **unknown word token** is added to the vocabulary.

It is written as:

$\langle \text{UNK} \rangle$

Words that are not present in the vocabulary are represented using the $\langle \text{UNK} \rangle$ token.

For example, if a rare word is not among the selected 10,000 vocabulary words, it is mapped to $\langle \text{UNK} \rangle$.

The lecture states that unknown words will be discussed in more detail later.

21. Important Distinctions

Training Example Index

Parentheses indicate the training example:

$x^{(i)}$

This refers to the input of training example i .

Sequence Position

Angle brackets indicate the position inside a sequence:

$x\langle t \rangle$

This refers to the input at position t .

Combined Notation

The notation:

$x^{(i)}\langle t \rangle$ or $x^{(i)}\langle t \rangle$

means the input at position t in training example i .

Similarly:

$y^{(i)}(t)$

means the output at position t in training example i .

Input Length and Output Length

T_x represents the input sequence length.

T_y represents the output sequence length.

They are equal in the named-entity recognition example, but they may be different in other tasks.

Sequence Position and Vocabulary Position

A word's position in a sentence is different from its position in the vocabulary.

For example:

- "Harry" appears at sequence position 1.
- "Harry" appears at vocabulary position 4075.

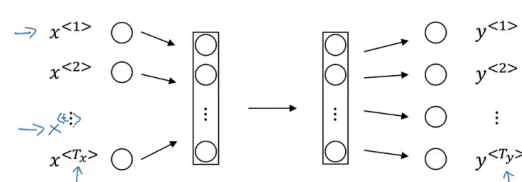
These positions must not be confused.

22. Key Takeaways

1. Sequence models process ordered sequences such as words in a sentence.
2. Named-entity recognition identifies entities such as people, companies, times, locations, countries, and currencies.
3. A simple named-entity recognition model can produce one binary output for each input word.
4. $x(t)$ represents the input at position t in a sequence.
5. $y(t)$ represents the output at position t in a sequence.
6. T_x represents the length of the input sequence.
7. T_y represents the length of the output sequence.
8. The input and output sequence lengths can be equal or different.
9. $x^{(i)}$ represents the input of training example i .
10. $x^{(i)}(t)$ represents the input at position t in training example i .
11. Different training examples can have different sequence lengths.
12. A vocabulary is a list of words that the model can represent.
13. Each word in the vocabulary is assigned a unique index.
14. A one-hot vector contains exactly one value equal to 1 and zeros everywhere else.
15. The dimension of a one-hot vector is equal to the vocabulary size.
16. A sequence model learns a mapping from X to Y using supervised learning.
17. Words that do not appear in the vocabulary are represented using the $\langle \text{UNK} \rangle$ token.

Recurrent Neural Network Model (RNN)

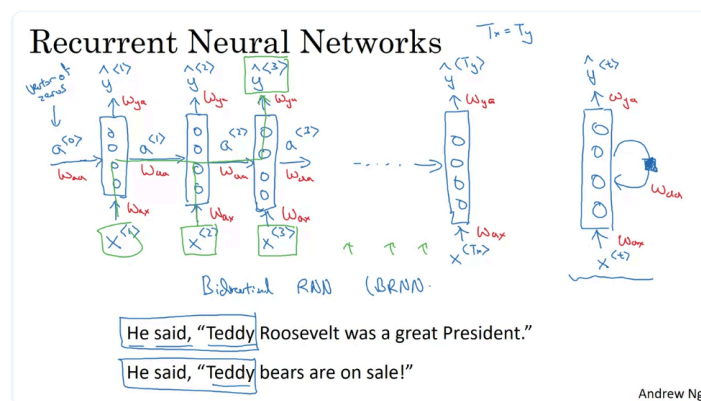
Why not a standard network?



Problems:

- Inputs, outputs can be different lengths in different examples.
- Doesn't share features learned across different positions of text.

some other position x t also means that that might be a person's name.



Recurrent Neural Networks

1. Learning the Mapping from X to Y

The previous part introduced the notation used to define sequence learning problems.

The next goal is to build a neural network that learns the mapping:

$$X \rightarrow Y$$

For the named-entity recognition example:

- X is the sequence of input words.
- Y is the sequence of labels indicating whether each word is part of a person's name.

The example sentence contains nine input words, represented by nine one-hot vectors.

The model must produce nine output values, where each output is either:

- 1 if the corresponding word is part of a person's name
- 0 if the corresponding word is not part of a person's name

2. Why Not Use a Standard Neural Network?

One possible approach is to use a standard feedforward neural network.

The model could:

1. Receive all nine one-hot vectors as input.
2. Pass them through several hidden layers.
3. Produce nine output values.

However, this approach does not work well for sequence data.

There are two main problems.

3. Problem 1: Variable Input and Output Lengths

Different training examples can have different input and output sequence lengths.

For example:

- One sentence may contain 9 words.
- Another sentence may contain 15 words.
- A different sentence may contain another number of words.

Therefore, T_x and T_y can vary between training examples.

A standard neural network usually expects a fixed-size input and a fixed-size output.

One possible solution would be to define a maximum sentence length and pad shorter sentences with zeros until they reach that length.

This process is called **zero-padding**.

However, padding every sequence to the maximum length still does not provide a good representation for the problem.

4. Problem 2: No Feature Sharing Across Sequence Positions

A standard neural network does not naturally share features learned at different positions in a sequence.

Suppose the model learns that the word "Harry" appearing at position 1 is likely to be part of a person's name.

It would be useful if the model could automatically apply the same knowledge when "Harry" appears at another position, such as position t .

A naive feedforward neural network does not naturally reuse the same learned feature across different sequence positions.

This is similar to the motivation for convolutional neural networks.

In a convolutional neural network, a feature learned in one part of an image can also be applied to other parts of the image.

For sequence data, it is also useful for a feature learned at one sequence position to generalize to other positions.

5. Large Number of Parameters

Each word in the example is represented by a 10,000-dimensional one-hot vector.

If a standard neural network receives all words at once, the total input size would be:

maximum number of words $\times$ 10,000

The weight matrix in the first layer would therefore contain a very large number of parameters.

Using a better architecture and representation can reduce the number of parameters.

A recurrent neural network avoids both major disadvantages:

- It can process sequences with varying lengths.
- It shares parameters across different sequence positions.

6. Basic Recurrent Neural Network Structure

A recurrent neural network, or **RNN**, processes a sequence one element at a time.

For a sentence read from left to right, the first input is:

$x\langle 1 \rangle$

The model passes $x\langle 1 \rangle$ into a neural network layer.

The network computes a hidden activation:

$a\langle 1 \rangle$

It may also produce a prediction:

$\hat{y}\langle 1 \rangle$

The prediction indicates whether the first word is part of a person's name.

7. Passing Information Between Time Steps

At the second time step, the RNN receives:

$x\langle 2 \rangle$

However, it does not compute $\hat{y}\langle 2 \rangle$ using only $x\langle 2 \rangle$.

It also receives information from the previous time step through the previous hidden activation:

$a\langle 1 \rangle$

The network uses both:

- $x\langle 2 \rangle$
- $a\langle 1 \rangle$

to compute:

- $a\langle 2 \rangle$
- $\hat{y}\langle 2 \rangle$

The activation $a\langle 1 \rangle$ allows information from the first word to influence the processing of the second word.

8. Processing the Entire Sequence

At the third time step, the network receives:

- $x\langle 3 \rangle$
- $a\langle 2 \rangle$

It then computes:

- $a\langle 3 \rangle$
- $\hat{y}\langle 3 \rangle$

This process continues until the final time step.

At the final input position, the network receives:

$x\langle T_x \rangle$

and produces a final prediction such as:

$\hat{y}\langle T_y \rangle$

In the named-entity recognition example:

$T_x = T_y$

However, the architecture must be modified for tasks where:

$T_x \neq T_y$

9. Hidden State

At every time step, the recurrent neural network passes an activation to the next time step.

This activation is also called the **hidden state**.

The hidden state at time t is written as:

$a\langle t \rangle$

It contains information gathered from the sequence up to time step t .

For example:

- $a\langle 1 \rangle$ contains information from $x\langle 1 \rangle$.
- $a\langle 2 \rangle$ contains information from $x\langle 1 \rangle$ and $x\langle 2 \rangle$.
- $a\langle 3 \rangle$ contains information from $x\langle 1 \rangle$, $x\langle 2 \rangle$, and $x\langle 3 \rangle$.

This information is passed forward through the recurrent connections.

10. Initial Hidden State

Before processing the first input, the RNN requires an initial hidden state:

$a\langle 0 \rangle$

The most common choice is to initialize $a\langle 0 \rangle$ as a vector of zeros.

Conceptually:

$a\langle 0 \rangle = 0$

Some researchers initialize $a\langle 0 \rangle$ randomly, and other initialization methods are also possible.

However, using a zero vector is the most common choice.

The first hidden state is therefore computed using:

- $x\langle 1 \rangle$
- $a\langle 0 \rangle$

11. Unrolled RNN Diagram

An RNN can be represented as an **unrolled diagram**.

In the unrolled representation, each time step is shown separately:

$x\langle 1 \rangle \rightarrow a\langle 1 \rangle \rightarrow \hat{y}\langle 1 \rangle$

$x\langle 2 \rangle \rightarrow a\langle 2 \rangle \rightarrow \hat{y}\langle 2 \rangle$

$x\langle 3 \rangle \rightarrow a\langle 3 \rangle \rightarrow \hat{y}\langle 3 \rangle$

...

$x\langle T_x \rangle \rightarrow a\langle T_x \rangle \rightarrow \hat{y}\langle T_y \rangle$

The hidden states are connected horizontally:

$a\langle 0 \rangle \rightarrow a\langle 1 \rangle \rightarrow a\langle 2 \rangle \rightarrow a\langle 3 \rangle \rightarrow \dots$

This diagram makes it clear how information passes from one time step to the next.

12. Recurrent Loop Diagram

Some books and research papers represent an RNN using a more compact diagram.

In this representation:

- The network layer is drawn once.
- A loop points from the layer back to itself.
- The loop represents the recurrent connection.
- A shaded box may represent a delay of one time step.

The compact recurrent diagram and the unrolled diagram represent the same computation.

A useful way to interpret the compact diagram is to mentally unroll it across time steps.

The lecture uses the unrolled representation because it is easier to interpret.

13. Left-to-Right Processing

The basic RNN scans the sequence from left to right.

At time step t , the model has processed:

$x\langle 1 \rangle, x\langle 2 \rangle, \dots, x\langle t \rangle$

It has not yet processed:

$x\langle t+1 \rangle, x\langle t+2 \rangle, \dots, x\langle T_x \rangle$

Therefore, the prediction at time t is based only on the current input and earlier inputs.

14. Parameter Sharing

The same parameters are used at every time step.

This is one of the most important properties of an RNN.

The parameter that controls the connection from the input to the hidden state is written as:

W_{ax}

The same W_{ax} is used for:

- $x\langle 1 \rangle$
- $x\langle 2 \rangle$
- $x\langle 3 \rangle$
- Every other input position

The parameter that controls the recurrent connection between hidden states is written as:

W_{aa}

The same W_{aa} is used for every transition:

- $a\langle 0 \rangle \rightarrow a\langle 1 \rangle$
- $a\langle 1 \rangle \rightarrow a\langle 2 \rangle$
- $a\langle 2 \rangle \rightarrow a\langle 3 \rangle$
- Every later transition

The parameter that controls the prediction from the hidden state is written as:

W_{ya}

The same W_{ya} is used to compute the output prediction at every time step.

15. Shared Parameters Across Time

The RNN uses the same parameter sets at all time steps:

- W_{ax} for input-to-hidden connections
- W_{aa} for hidden-to-hidden connections
- W_{ya} for hidden-to-output connections

These parameters are not different for each sequence position.

For example, there is not one separate W_{ax} for $x\langle 1 \rangle$ and another W_{ax} for $x\langle 2 \rangle$.

The same matrix is reused throughout the sequence.

This allows features learned at one position to be applied at every other position.

It also reduces the total number of parameters.

16. Information Used to Predict $\hat{y}\langle 3 \rangle$

When the RNN predicts:

$\hat{y}\langle 3 \rangle$

it uses information from the current input:

$x\langle 3 \rangle$

It also uses information from earlier inputs through the previous hidden state:

$a\langle 2 \rangle$

Because $a\langle 2 \rangle$ was influenced by $x\langle 1 \rangle$ and $x\langle 2 \rangle$, the prediction $\hat{y}\langle 3 \rangle$ can depend on:

- $x\langle 1 \rangle$
- $x\langle 2 \rangle$
- $x\langle 3 \rangle$

Information from $x\langle 1 \rangle$ can travel through the hidden states:

$x\langle 1 \rangle \rightarrow a\langle 1 \rangle \rightarrow a\langle 2 \rangle \rightarrow a\langle 3 \rangle$

This information can then help produce $\hat{y}\langle 3 \rangle$.

17. Limitation of a Unidirectional RNN

A basic RNN only uses information from earlier positions in the sequence.

When predicting $\hat{y}\langle 3 \rangle$, it can use information from:

- $x\langle 1 \rangle$
- $x\langle 2 \rangle$
- $x\langle 3 \rangle$

However, it cannot use information from later inputs such as:

- $x\langle 4 \rangle$
- $x\langle 5 \rangle$
- $x\langle 6 \rangle$

This can be a serious limitation.

18. Example: "Teddy Roosevelt"

Consider the sentence:

He said Teddy Roosevelt was a great president.

To determine whether “Teddy” is part of a person's name, it is helpful to see the later word “Roosevelt.”

In this sentence, “Teddy” is part of the person's name “Teddy Roosevelt.”

Now consider another sentence:

He said teddy bears are on sale.

In this sentence, “teddy” is not part of a person's name.

If the model only looks at the first three words:

He said Teddy

it cannot determine with certainty whether “Teddy” is part of a person's name.

The later words provide important information:

- “Roosevelt” suggests that “Teddy” is a person's name.
- “bears” suggests that “teddy” is not a person's name.

19. Need for Future Context

The prediction at a given sequence position may depend on both:

- Earlier words
- Later words

A unidirectional RNN only has access to earlier words when making a prediction.

Therefore, it cannot use future context.

This is a limitation for tasks such as named-entity recognition, where later words can change the interpretation of an earlier word.

20. Bidirectional Recurrent Neural Networks

The limitation of using only earlier information can be addressed using a **Bidirectional Recurrent Neural Network**, commonly abbreviated as **BRNN**.

A bidirectional RNN allows a prediction to use:

- Information from earlier positions
- Information from later positions

For example, the prediction $\hat{y}\langle 3 \rangle$ could use information from both:

- $x\langle 1 \rangle$ and $x\langle 2 \rangle$
- $x\langle 4 \rangle$, $x\langle 5 \rangle$, and later inputs

Bidirectional RNNs are introduced in a later video.

For now, the simpler unidirectional RNN is sufficient for explaining the main concepts.

21. Unidirectional RNN Summary

A basic unidirectional RNN:

1. Processes the sequence from left to right.
2. Receives one input element at each time step.
3. Passes a hidden state from one time step to the next.
4. Uses shared parameters across all sequence positions.
5. Produces an output prediction at each time step.
6. Uses only the current and previous inputs.
7. Does not use information from later positions.

22. Main Advantages of an RNN

Compared with a standard feedforward neural network, an RNN provides several advantages.

Variable-Length Sequences

An RNN can process input sequences of different lengths.

It does not require every sentence to contain the same number of words.

Parameter Sharing

The same parameters are reused at every time step.

This allows the model to apply the same learned features at different sequence positions.

Fewer Parameters

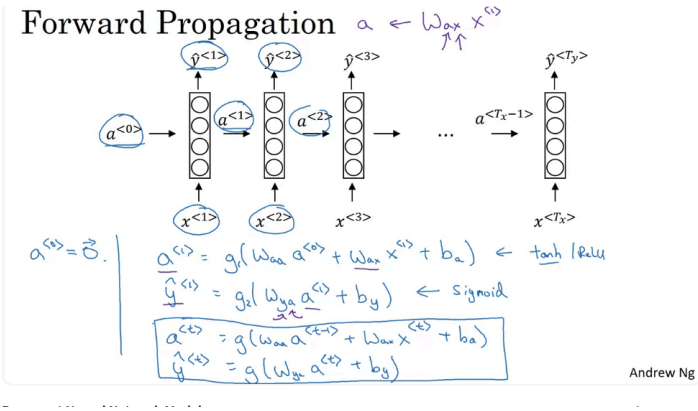
Because the same weight matrices are reused, the number of parameters does not grow directly with the maximum sequence length.

Context from Previous Inputs

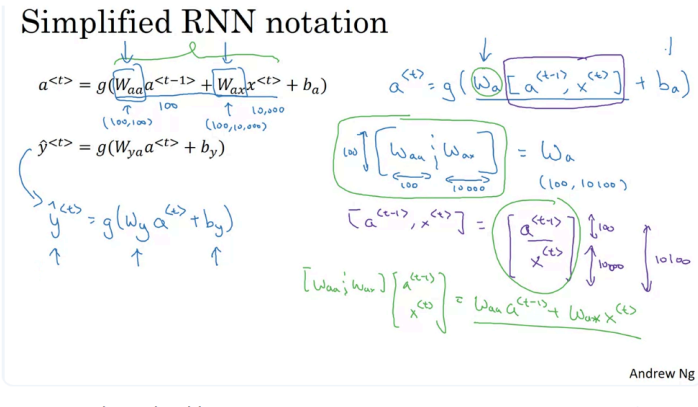
The hidden state allows information from earlier inputs to influence later predictions.

23. Important Notation

Notation	Meaning
$x(t)$	Input at time step t
$a(t)$	Hidden activation at time step t
$\hat{y}(t)$	Predicted output at time step t
$a(0)$	Initial hidden state
T_x	Input sequence length
T_y	Output sequence length
W_{ax}	Input-to-hidden parameter matrix
W_{aa}	Hidden-to-hidden recurrent parameter matrix
W_{ya}	Hidden-to-output parameter matrix



Concatenate Waa with Wax:



So we just need to manage 1 Weight matrix instead of 2 Weight matrices

Forward Propagation in a Recurrent Neural Network

1. Initial Hidden State

A basic recurrent neural network usually begins with an initial hidden state:

$$a\langle 0 \rangle$$

The most common initialization is a vector containing only zeros:

$$a\langle 0 \rangle = 0$$

This initial state is used together with the first input $x\langle 1 \rangle$ to compute the first hidden activation.

2. Computing the First Hidden State

At the first time step, the RNN computes $a\langle 1 \rangle$ using:

- The previous hidden state $a\langle 0 \rangle$
- The current input $x\langle 1 \rangle$
- The recurrent weight matrix W_{aa}
- The input weight matrix W_{ax}
- The activation bias b_a
- An activation function g

The computation is:

$$a\langle 1 \rangle = g(W_{aa}a\langle 0 \rangle + W_{ax}x\langle 1 \rangle + b_a)$$

The hidden state $a\langle 1 \rangle$ combines information from the initial hidden state and the first input.

3. Computing the First Prediction

After computing $a\langle 1 \rangle$, the RNN computes the first output prediction:

$$\hat{y}\langle 1 \rangle = g(W_{ya}a\langle 1 \rangle + b_y)$$

Here:

- W_{ya} maps the hidden state to the output.
- b_y is the output bias.
- g is an output activation function.

The activation function used for the output does not necessarily have to be the same as the activation function used for the hidden state.

4. General Forward-Propagation Equations

At any time step t , the hidden state is computed as:

$$a\langle t \rangle = g(W_{aa}a\langle t-1 \rangle + W_{ax}x\langle t \rangle + b_a)$$

The output prediction is computed as:

$$\hat{y}\langle t \rangle = g(W_{ya}a\langle t \rangle + b_y)$$

These equations describe the forward-propagation process of a basic RNN.

At every time step:

1. The current input $x\langle t \rangle$ is received.
2. The previous hidden state $a\langle t-1 \rangle$ is received.
3. The new hidden state $a\langle t \rangle$ is computed.
4. The output prediction $\hat{y}\langle t \rangle$ is computed.

5. Forward Propagation Through the Sequence

Forward propagation proceeds from left to right through the sequence.

The process begins with:

$a\langle 0 \rangle = 0$

Then:

- $a\langle 0 \rangle$ and $x\langle 1 \rangle$ are used to compute $a\langle 1 \rangle$.
- $a\langle 1 \rangle$ is used to compute $\hat{y}\langle 1 \rangle$.
- $a\langle 1 \rangle$ and $x\langle 2 \rangle$ are used to compute $a\langle 2 \rangle$.
- $a\langle 2 \rangle$ is used to compute $\hat{y}\langle 2 \rangle$.
- The same process continues through the remaining time steps.

Conceptually:

$a\langle 0 \rangle, x\langle 1 \rangle \rightarrow a\langle 1 \rangle \rightarrow \hat{y}\langle 1 \rangle$

$a\langle 1 \rangle, x\langle 2 \rangle \rightarrow a\langle 2 \rangle \rightarrow \hat{y}\langle 2 \rangle$

$a\langle 2 \rangle, x\langle 3 \rangle \rightarrow a\langle 3 \rangle \rightarrow \hat{y}\langle 3 \rangle$

This continues until the final element in the sequence has been processed.

6. Meaning of the Weight-Matrix Subscripts

The subscripts of the weight matrices indicate:

1. The type of quantity used as input to the matrix
2. The type of quantity being computed

For example:

W_{ax}

The second subscript, x , indicates that the matrix multiplies an x -type quantity.

The first subscript, a , indicates that the matrix contributes to the computation of an a -type quantity.

Therefore, W_{ax} maps an input vector x to a hidden activation a .

Similarly:

W_{aa}

The second a indicates that the matrix multiplies a hidden activation.

The first a indicates that the result contributes to the computation of another hidden activation.

Therefore, W_{aa} maps the previous hidden state to the current hidden state.

For:

W_{ya}

The second subscript, a , indicates that the matrix multiplies an a -type quantity.

The first subscript, y , indicates that the matrix contributes to the computation of a y -type quantity.

Therefore, W_{ya} maps the hidden state to the output.

7. Parameter Interpretation

Parameter	Meaning
W_{ax}	Maps the current input $x\langle t \rangle$ to the hidden state $a\langle t \rangle$
W_{aa}	Maps the previous hidden state $a\langle t-1 \rangle$ to the current hidden state $a\langle t \rangle$
W_{ya}	Maps the hidden state $a\langle t \rangle$ to the output prediction $\hat{y}\langle t \rangle$
b_a	Bias used when computing the hidden state
b_y	Bias used when computing the output prediction

8. Hidden-State Activation Function

The activation function used to compute the hidden state is commonly:

$\tanh$

Therefore, a common RNN hidden-state equation is:

$$a(t) = \tanh(W_{aa}a(t-1) + W_{ax}x(t) + b_a)$$

The lecture mentions that ReLU is also sometimes used.

However, $\tanh$ is a common activation-function choice for a basic RNN.

The vanishing-gradient problem and methods for addressing it are discussed later in the course.

9. Output Activation Function

The activation function used to compute $\hat{y}(t)$ depends on the type of output.

Binary Classification

For a binary classification problem, the output can use a sigmoid activation function.

For example, named-entity recognition may use:

- 1 if the word is part of a person's name
- 0 if the word is not part of a person's name

In this case:

$$\hat{y}(t) = \text{sigmoid}(W_{ya}a(t) + b_y)$$

Multi-Class Classification

For a classification problem with k possible classes, the output can use a softmax activation function.

In this case, $\hat{y}(t)$ represents a probability distribution over the k classes.

Therefore, the choice of output activation depends on the structure of y .

10. Different Activation Functions

The activation function used for the hidden state and the activation function used for the output may be different.

For example:

- $\tanh$ may be used for $a(t)$.
- sigmoid may be used for $\hat{y}(t)$ in binary classification.
- softmax may be used for $\hat{y}(t)$ in multi-class classification.

The two activation functions could be written as g_1 and g_2 to distinguish them.

However, the lecture often uses g for both to keep the notation simple.

11. Simplifying the Hidden-State Equation

The original hidden-state equation uses two separate weight matrices:

$$a(t) = g(W_{aa}a(t-1) + W_{ax}x(t) + b_a)$$

This equation can be written more compactly by combining:

- W_{aa}
- W_{ax}

into a single matrix:

W_a

The vectors $a(t-1)$ and $x(t)$ are also combined into one stacked vector.

The simplified equation is:

$$a(t) = g(W_a[a(t-1), x(t)] + b_a)$$

Here, $[a(t-1), x(t)]$ means that the two vectors are stacked together.

12. Constructing the Combined Matrix W_a

The combined matrix W_a is created by placing W_{aa} and W_{ax} side by side:

$$W_a = [W_{aa} \ W_{ax}]$$

This is a horizontal concatenation of the two matrices.

The combined matrix performs both transformations in one matrix multiplication.

Therefore:

$$W_a[a(t-1), x(t)]$$

is equivalent to:

$$W_{aa}a(t-1) + W_{ax}x(t)$$

The simplified notation does not change the computation. It only makes the equations more compact.

13. Stacking the Input Vectors

The notation:

$$[a(t-1), x(t)]$$

means that $a(t-1)$ and $x(t)$ are stacked into one vector.

Conceptually:

$$[a(t-1)$$

$$x(t)]$$

If $a(t-1)$ has 100 elements and $x(t)$ has 10,000 elements, the stacked vector has:

$$100 + 10,000 = 10,100 \text{ elements}$$

Therefore:

$$[a(t-1), x(t)] \in \mathbb{R}^{10100}$$

14. Example Matrix Dimensions

Suppose the hidden state has 100 dimensions:

$$a(t) \in \mathbb{R}^{100}$$

Suppose the input is a 10,000-dimensional one-hot vector:

$$x(t) \in \mathbb{R}^{10000}$$

Then:

$$W_{aa} \in \mathbb{R}^{100 \times 100}$$

because it maps a 100-dimensional hidden state to another 100-dimensional hidden state.

Also:

$$W_{ax} \in \mathbb{R}^{100 \times 10000}$$

because it maps a 10,000-dimensional input to a 100-dimensional hidden state.

After horizontal concatenation:

$$W_a = [W_{aa} \ W_{ax}]$$

The dimension of W_a is:

$$W_a \in \mathbb{R}^{100 \times 10100}$$

The stacked vector has dimension:

$$[a(t-1), x(t)] \in \mathbb{R}^{10100}$$

Therefore, the matrix multiplication produces a 100-dimensional vector:

$$W_a[a(t-1), x(t)] \in \mathbb{R}^{100}$$

15. Verifying the Combined Computation

The compact matrix multiplication can be expanded as:

$$[W_{aa} \ W_{ax}][a(t-1), x(t)]$$

This produces:

$$W_{aa}a(t-1) + W_{ax}x(t)$$

Therefore:

$$W_a[a(t-1), x(t)] = W_{aa}a(t-1) + W_{ax}x(t)$$

The original and simplified forms are mathematically equivalent.

Original form

$$a(t) = g(W_{aa}a(t-1) + W_{ax}x(t) + b_a)$$

Simplified form

$$a(t) = g(W_a[a(t-1), x(t)] + b_a)$$

The simplified form is useful when developing more complex sequence models.

16. Simplifying the Output Equation

The original output equation is:

$$\hat{y}(t) = g(W_{ya}a(t) + b_y)$$

The notation can be shortened by renaming W_{ya} as W_y :

$$W_y = W_{ya}$$

The output equation becomes:

$$\hat{y}(t) = g(W_ya(t) + b_y)$$

The shorter notation indicates that W_y is a weight matrix used to compute a y-type output.

17. Compact RNN Equations

Using the simplified notation, the forward-propagation equations are:

$$a(t) = g(W_a[a(t-1), x(t)] + b_a)$$

$$\hat{y}(t) = g(W_ya(t) + b_y)$$

The parameters are:

- W_a
- b_a
- W_y
- b_y

The first equation computes an activation-like quantity.

The second equation computes a y-like output quantity.

18. Original and Compact Notation

Original notation	Compact notation
W_{aa} and W_{ax}	W_a
W_{ya}	W_y
$W_{aa}a(t-1) + W_{ax}x(t)$	$W_a[a(t-1), x(t)]$
$\hat{y}(t) = g(W_{ya}a(t) + b_y)$	$\hat{y}(t) = g(W_ya(t) + b_y)$

Both forms describe the same RNN.

The compact form is mainly used to reduce the amount of notation.

19. Important Dimension Summary

Using the example from the lecture:

Quantity	Dimension
$a\langle t-1 \rangle$	100
$x\langle t \rangle$	10,000
Stacked $[a\langle t-1 \rangle, x\langle t \rangle]$	10,100
W_{aa}	100×100
W_{ax}	$100 \times 10,000$
W_a	$100 \times 10,100$
Output of $W_a[a\langle t-1 \rangle, x\langle t \rangle]$	100

The dimensions are compatible because:

$$(100 \times 10,100)(10,100 \times 1) = 100 \times 1$$

20. Complete Forward-Propagation Procedure

For a sequence $x\langle 1 \rangle, x\langle 2 \rangle, \dots, x\langle T_x \rangle$:

Initialization

Set:

$$a\langle 0 \rangle = 0$$

At each time step t

Compute the hidden state:

$$a\langle t \rangle = g(W_{aa}a\langle t-1 \rangle + W_{ax}x\langle t \rangle + b_a)$$

or equivalently:

$$a\langle t \rangle = g(W_a[a\langle t-1 \rangle, x\langle t \rangle] + b_a)$$

Then compute the prediction:

$$\hat{y}\langle t \rangle = g(W_{ya}a\langle t \rangle + b_y)$$

or equivalently:

$$\hat{y}\langle t \rangle = g(W_y a\langle t \rangle + b_y)$$

Repeat this process from $t = 1$ to $t = T_x$.

21. Key Takeaways

1. The initial hidden state $a\langle 0 \rangle$ is usually initialized as a zero vector.
2. The hidden state at time t depends on both the previous hidden state and the current input.
3. The basic hidden-state equation is:
$$a\langle t \rangle = g(W_{aa}a\langle t-1 \rangle + W_{ax}x\langle t \rangle + b_a)$$
4. The prediction at time t is computed from the current hidden state:
$$\hat{y}\langle t \rangle = g(W_{ya}a\langle t \rangle + b_y)$$
5. $\tanh$ is a common activation function for the hidden state.
6. ReLU may also be used for hidden-state activations.
7. Sigmoid can be used for binary output classification.
8. Softmax can be used for k -class output classification.
9. The hidden-state and output activation functions can be different.
10. The subscripts of a weight matrix describe the input quantity and the output quantity associated with that matrix.
11. W_{aa} and W_{ax} can be horizontally combined into one matrix W_a .
12. $a\langle t-1 \rangle$ and $x\langle t \rangle$ can be stacked into one vector.

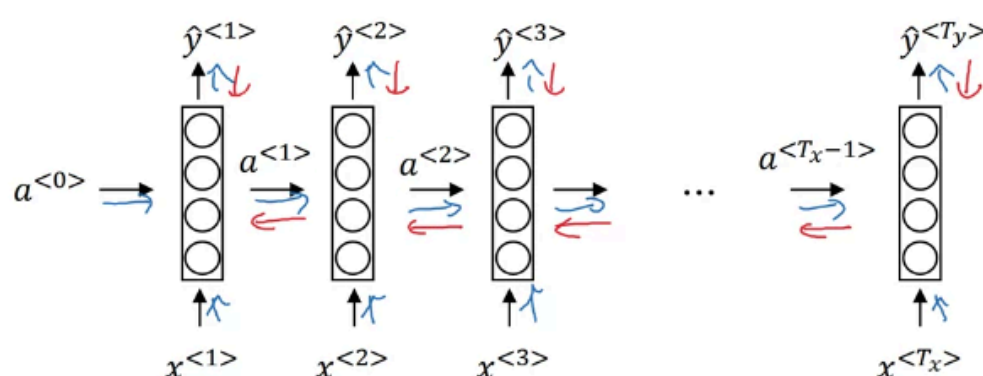
13. The compact equation is mathematically equivalent to the original equation.
14. W_{ya} can be shortened to W_y .
15. Compact notation is useful for describing more complex neural-network architectures.

22. End of the Basic RNN Introduction

At this point, the basic structure and forward-propagation calculations of a recurrent neural network have been introduced. The next topic is **backpropagation in recurrent neural networks**, which explains how the RNN parameters are learned.

Backpropagation Through Time

Forward propagation and backpropagation

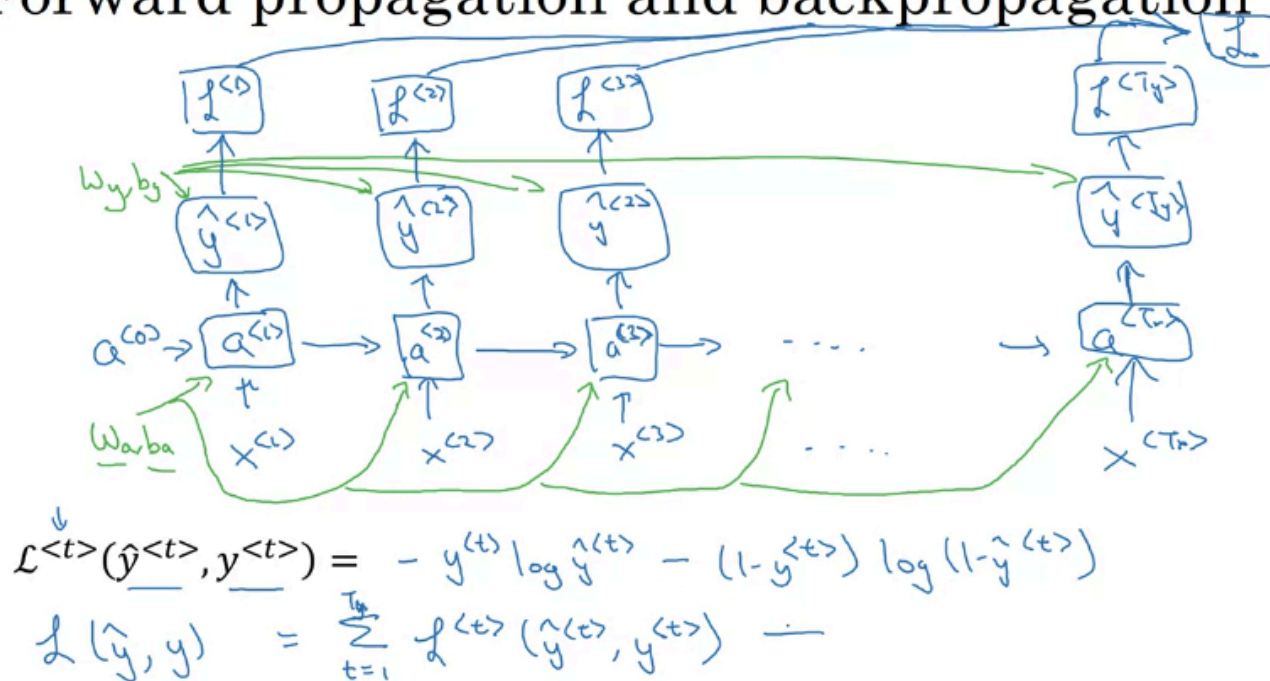


Andrew Ng

Forward flow is like the blue arrows, and the backward flow is like the red one

For more detail:

Forward propagation and backpropagation

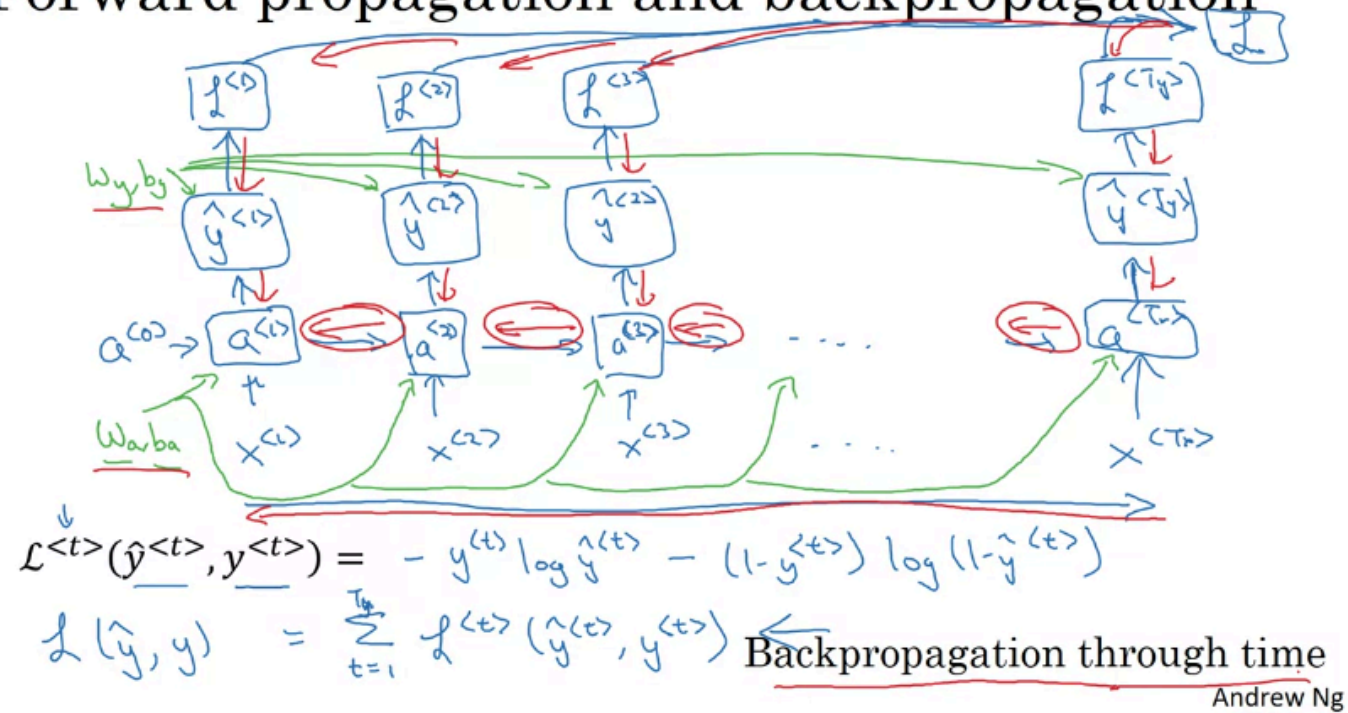


Andrew Ng

In forward pass: we use same parameter W_{aba} to compute $a^{<i>}$ and W_{by} to compute $y^{<i>}$

Then we compute loss from each output using foundation binary cross entropy, so, the final loss function is the sum of all loss in each output.

Forward propagation and backpropagation



Andrew Ng

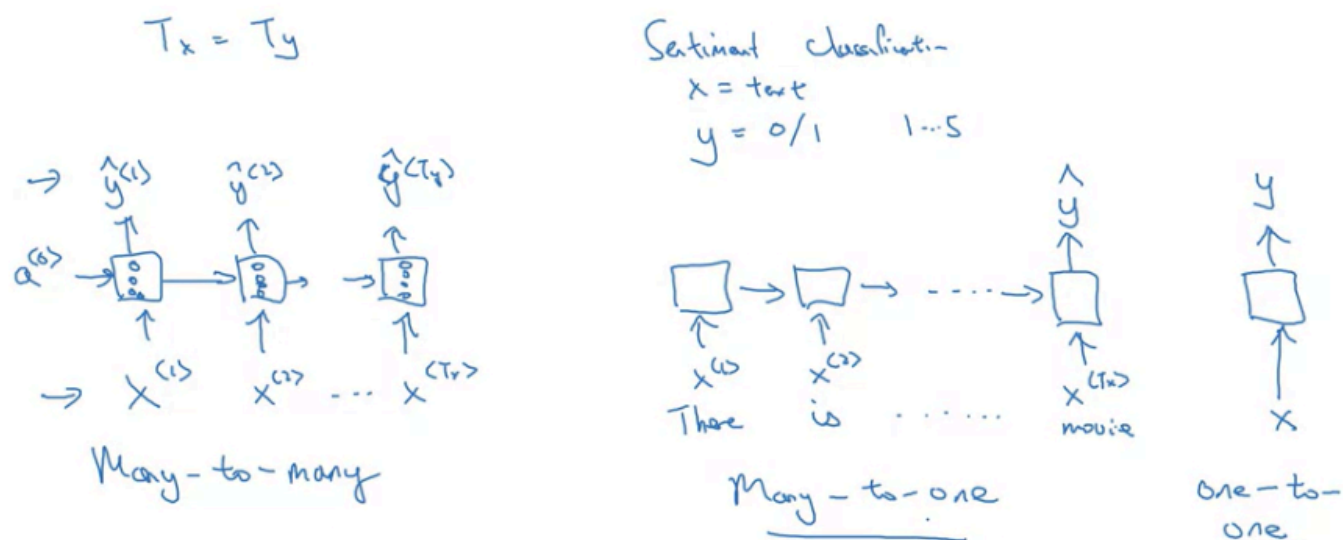
Update weight using gradient descent → the red narrows show the flow of the gradient: Total loss → each loss → a and y

Backpropagation through time

The motivation for this name is that for forward prop, you are scanning from left to right, increasing indices of the time, t , whereas, the backpropagation, you're going from right to left, you're kind of going backwards in time

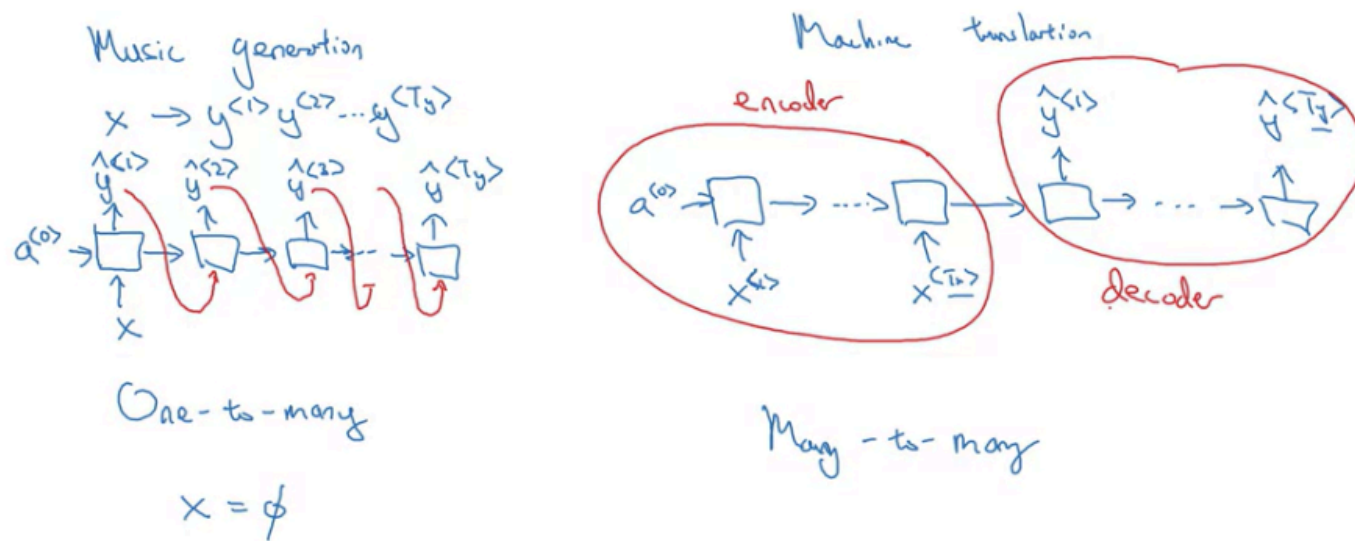
Different Types of RNNs

Examples of RNN architectures



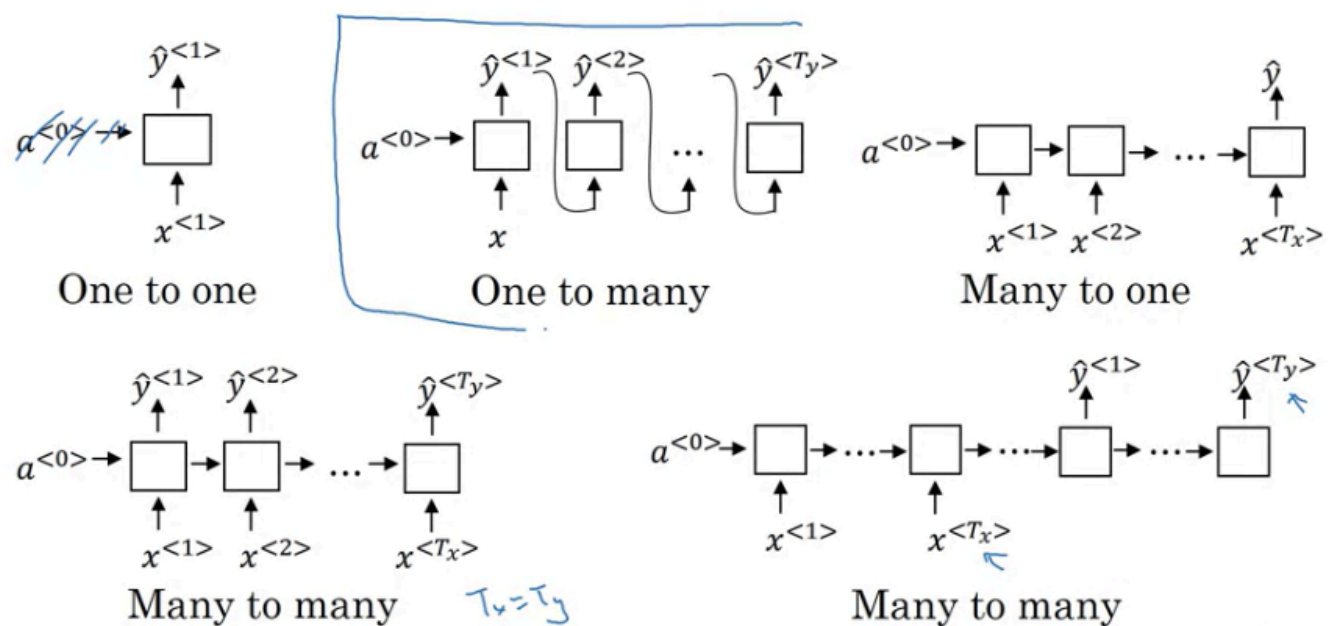
Andrew Ng

Examples of RNN architectures



So this would be an example of a many-to-many architecture. Andrew Ng

Summary of RNN types



Andrew Ng

Types of RNN Architectures

Recurrent Neural Networks can be organized according to the number of inputs and outputs they process.

1. One-to-One

Structure:

One input $\rightarrow$ One output

This is equivalent to a standard feedforward neural network. Since no sequence is involved, an RNN is usually unnecessary.

Example:

Image classification:

Image $\rightarrow$ Class label

2. One-to-Many

Structure:

One input $\rightarrow$ A sequence of outputs

The model receives a single input and generates multiple outputs over several time steps.

Common applications:

- Music generation
- Text generation
- Sequence generation
- Image captioning

Example:

Initial input $\rightarrow y^1, y^2, y^3, \dots, y^{T_y}$

The generated output at one time step may also influence the next output.

3. Many-to-One

Structure:

A sequence of inputs $\rightarrow$ One output

The model processes the entire input sequence before producing a single prediction.

Example: Sentiment Classification

The input is a sequence of words from a movie review:

$x^1, x^2, x^3, \dots, x^{T_x} \rightarrow$ Positive or Negative

The model reads the complete review and predicts whether the reviewer liked the movie.

Other applications:

- Spam detection
- Topic classification
- Emotion recognition
- Audio classification

4. Many-to-Many with Equal Sequence Lengths

Structure:

A sequence of inputs $\rightarrow$ A sequence of outputs

In this architecture, the input and output sequences have the same length:

$T_x = T_y$

Each input time step corresponds to an output time step.

Example: Named Entity Recognition

Input:

Harry visited New York

Output:

Person, Other, Location, Location

Each word receives its own label.

Other applications:

- Part-of-speech tagging
- Sequence labeling
- Frame-level audio classification

5. Many-to-Many with Different Sequence Lengths

Structure:

An input sequence $\rightarrow$ An output sequence with a potentially different length

In this case:

$T_x \neq T_y$

The model first processes the input sequence and then generates the output sequence.

Example: Machine Translation

English input:

Jane is visiting Africa

French output:

Jane visite l'Afrique

The number of words in the input and output sentences does not need to be equal.

This architecture is commonly implemented using an **encoder–decoder**, also called a **sequence-to-sequence model**.

Architecture Summary

Architecture	Input	Output	Example
One-to-One	Single value	Single value	Image classification
One-to-Many	Single value	Sequence	Music generation
Many-to-One	Sequence	Single value	Sentiment classification
Many-to-Many, $T_x = T_y$	Sequence	Sequence of equal length	Named Entity Recognition
Many-to-Many, $T_x \neq T_y$	Sequence	Sequence of different length	Machine translation

Key Takeaways

- RNN architectures differ according to the relationship between their input and output sequences.
- One-to-one problems usually do not require an RNN.
- Many-to-one architectures are suitable for sequence classification.
- One-to-many architectures are suitable for sequence generation.
- Many-to-many architectures are used for sequence labeling and sequence transformation.
- Machine translation requires a model that supports different input and output sequence lengths.
- Sequence generation introduces additional challenges that require special decoding techniques.

Notation

- T_x : Length of the input sequence
- T_y : Length of the output sequence
- x^t : Input at time step t
- y^t : Output at time step t
- $T_x = T_y$: Input and output sequences have equal lengths
- $T_x \neq T_y$: Input and output sequences have different lengths

Further Learning

Sequence-to-Sequence Models

Learn how encoder–decoder architectures transform one sequence into another, especially for machine translation.

Suggested topic:

- Sequence-to-Sequence Learning with Neural Networks
- Encoder–Decoder Architecture
- Context Vectors

Sequence Generation

Explore how RNNs generate text, music, or other sequential data.

Suggested topics:

- Autoregressive generation
- Greedy decoding
- Beam search
- Sampling with temperature

Named Entity Recognition

Study how many-to-many RNNs assign a label to every token in a sentence.

Suggested topics:

- BIO tagging format
- Bidirectional RNNs
- Conditional Random Fields
- Token classification

Improved RNN Architectures

Standard RNNs may struggle with long-term dependencies. More advanced architectures include:

- Long Short-Term Memory networks
- Gated Recurrent Units
- Bidirectional RNNs

Modern Sequence Models

For a broader understanding, compare RNNs with newer architectures:

- Attention mechanisms
- Transformers
- Encoder-only models such as BERT
- Encoder–decoder models such as T5
- Decoder-only models such as GPT

Language Model and Sequence Generation